Experiment 3

Implementation of Arithmetic Circuits Using FPGA

Objective

The objective of this experiment is to design and implement fundamental arithmetic circuits on an FPGA. These circuits include half adder, full adder, half subtractor, full subtractor, and a comparator. Each circuit will be designed using Verilog HDL and implemented on the FPGA using the specified pinout.

1. Half Adder

1.1. Theory

The half adder is a basic digital circuit used to add two single-bit binary numbers. It produces two outputs:

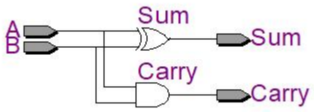
- Sum (S): The result of the XOR operation between the two inputs Sum = A $\oplus$ B
- Carry (C): The result of the AND operation between the two inputs Carry = A AND B

1.2. Structural Verilog Code

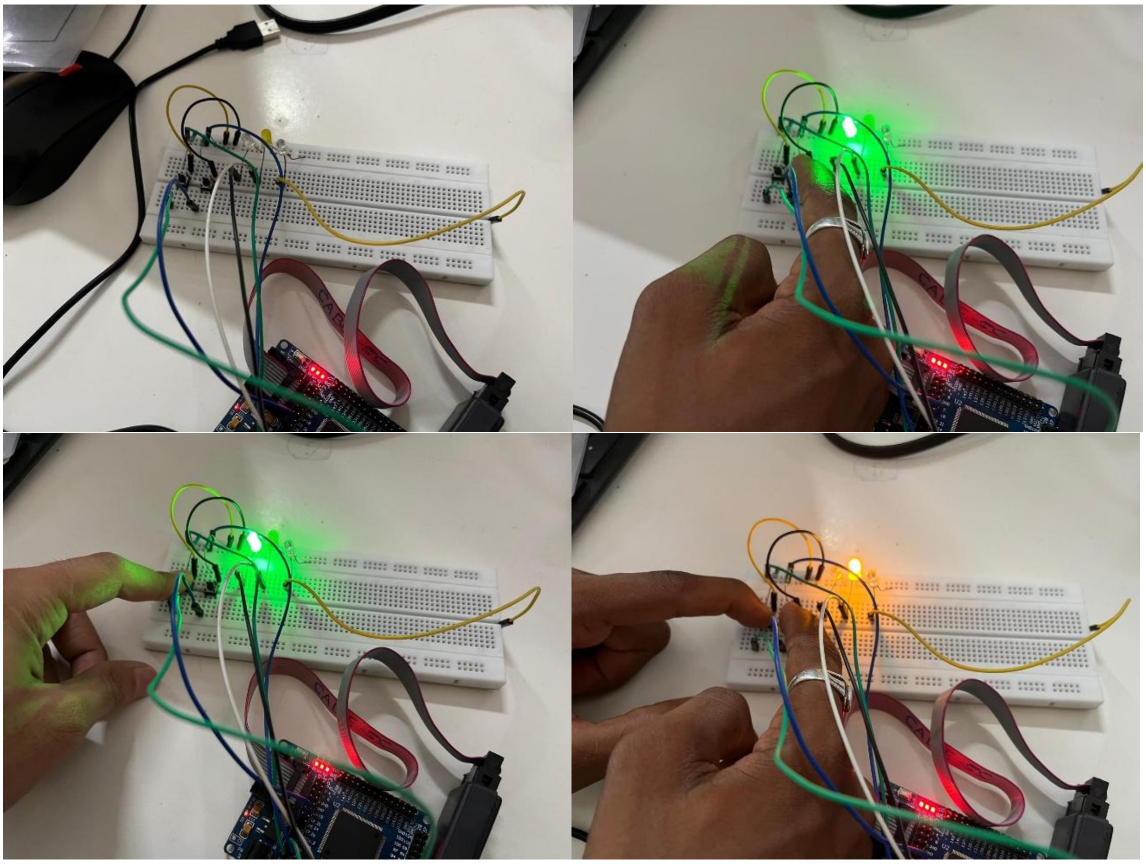
```
module Half_Adder(
   input A,
   input B,
   output Sum,
   output Carry
);

assign Sum   = A ^ B;
assign Carry = A & B;

endmodule
```

	Marwadi University **Faculty of Technology** **Department of Information and Communication Technology**	
Aim	**Implement adder circuits such as half/full adder on the FPGA development board.**	
Exp No. 3	**Date:**	**Enrollment No: 92301733025**

1.4 RTL Schematic:



1.5 Output



2. Full Adder

2.1. Theory

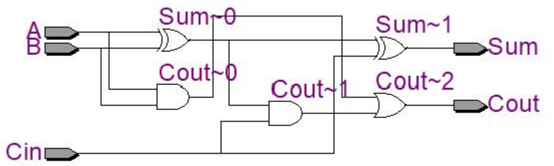
The full adder circuit adds three single-bit binary numbers, including a carry-in bit. It provides two outputs:

- Sum (S): Computed using SUM = (A XOR B) XOR Cin = $(A \oplus B) \oplus Cin$
- Carry Out (Cout): Computed using CARRY-OUT = A AND B OR Cin(A XOR B) = $A.B + Cin(A \oplus B)$.

2.2. Structural Verilog Code

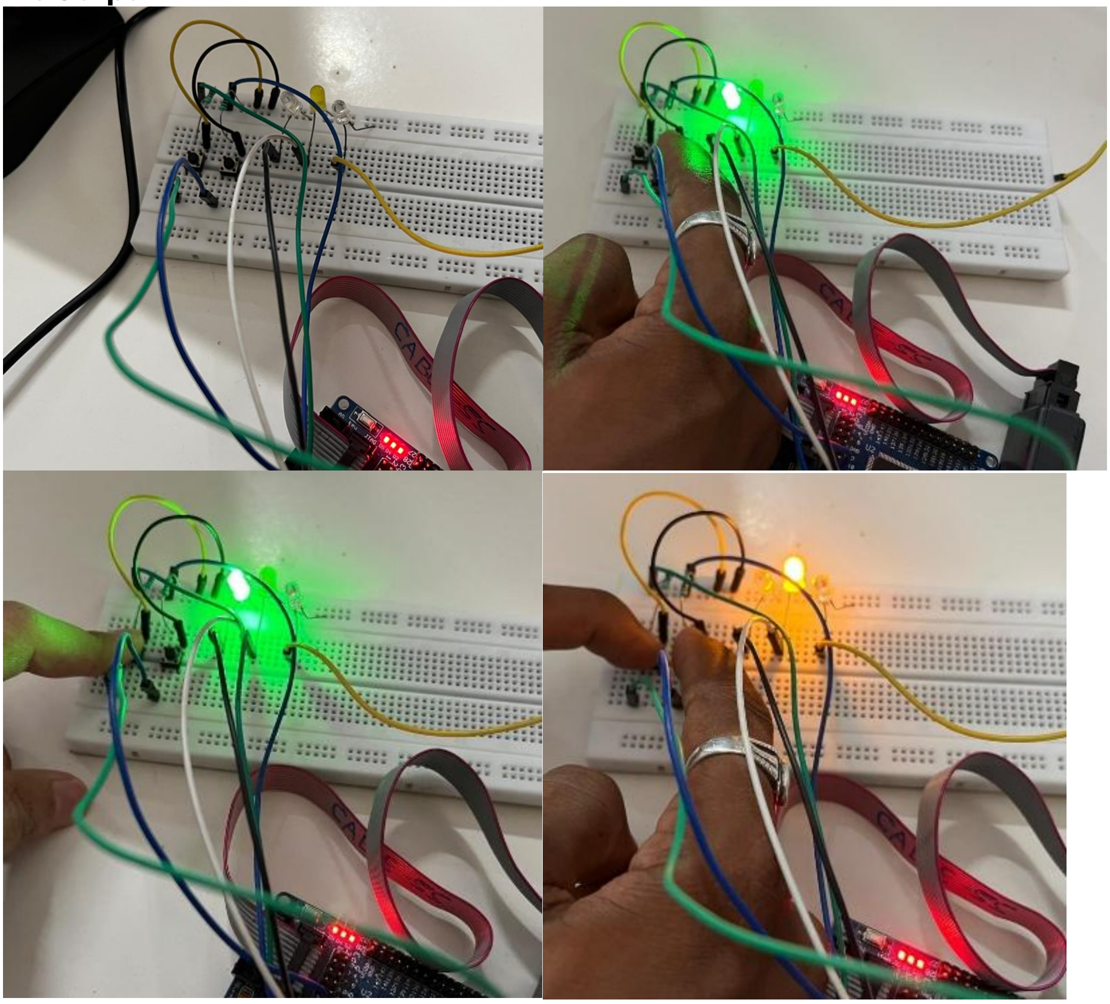
```
//--------------------------------------------------
// Full Adder
//--------------------------------------------------
module Full_Adder(
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
);

assign Sum  = A ^ B ^ Cin;
assign Cout = (A & B) | (Cin & (A ^ B));

endmodule
```

2.4 RTL Schematic:



	Marwadi University **Faculty of Technology** **Department of Information and Communication Technology**	
Aim	**Implement adder circuits such as half/full adder on the FPGA development board.**	
Exp No. 3	**Date:**	**Enrollment No: 92301733025**

2.5 Output



Conclusion

In this experiment, we designed and implemented Half Adder and Full Adder circuits in Verilog HDL on the FPGA development board, then synthesized, programmed, and tested them using Quartus II. The outputs matched the expected truth tables, confirming that both circuits operated correctly. The experiment covered digital arithmetic, FPGA implementation, Verilog coding, pin assignment, and hardware verification — practical groundwork for more complex digital systems.

Post Lab Exercise

1. **What is the function of a half adder?**
   A half adder is a combinational logic circuit that adds two single-bit binary inputs and produces two outputs: Sum (S) and Carry (C).

2. **Which logic gate is primarily used to generate the Sum output?**
   The **XOR (Exclusive-OR) gate** is primarily used to generate the **Sum** output.
   Sum = $A \oplus B$

3. **What is the main difference between a half adder and a full adder?**
   A Half Adder adds only two input bits and does not have a carry input, whereas a Full Adder adds three input bits (A, B, and Carry-in) and produces both Sum and Carry-out.

4. **Why is a full adder preferred in multi-bit addition?**
   A full adder is preferred because it accepts a **Carry-in** from the previous stage, allowing multiple full adders to be cascaded for performing multi-bit binary addition.

5. **What is the purpose of implementing adders on an FPGA?**
   Implementing adders on an FPGA allows designers to verify arithmetic circuits on real hardware, test their functionality, and build high-speed digital systems such as ALUs, processors, and digital signal processing circuits.

6. **Which FPGA resources are commonly used to display adder outputs?**
   The outputs of adders are commonly displayed using LEDs, seven-segment displays, and monitored through GPIO pins on the FPGA development board.

7. **What is the advantage of FPGA implementation over software simulation?**
   FPGA implementation allows **real-time hardware execution**, faster operation, and testing with actual input/output devices, whereas software simulation only verifies the design virtually without physical hardware interaction.